

Approach Document

SHL Conversational Assessment Recommender

This document summarises how I built the stateless FastAPI recommender for the SHL Individual Test Solutions catalog. The service takes multi-turn hiring-manager dialogue and returns a grounded shortlist (1-10 assessments) with catalog URLs only.

1. Problem framing

The core task is not keyword search. Users arrive with partial intent (role, level, constraints) and refine over several turns. The agent must clarify when context is thin, refuse off-scope legal advice, compare products using catalog descriptions, and update the shortlist when requirements change. Every recommendation URL is validated against a local whitelist so hallucinated products cannot appear in the API response.

2. Catalog preparation

I used the official `shl_product_catalog.json` feed. The raw file contains invalid embedded newlines inside string values, so I added a small sanitiser before JSON parse. Pre-packaged Job Solutions are excluded via name rules (* Solution, * Short Form, Job Focused Assessment), leaving 370 Individual Test Solutions. `test_type` is derived in code from catalog keys (for example Development & 360 maps to D; multi-key items use comma-separated letters) so the API matches the sample conversation format.

3. Retrieval design

Passing the full catalog to the LLM each turn is too slow and noisy. I built a hybrid retriever: sentence-transformers/all-MiniLM-L6-v2 embeddings with a FAISS inner-product index, plus a light keyword overlap boost and query expansion for domain terms (leadership, sales, Java, contact centre, etc.). The top 25 candidates are sent to the model. At Docker build time the index and embedding model are pre-baked so Render startup stays within 512 MB RAM.

4. Conversation policy

Claude Sonnet (claude-sonnet-4-20250514) handles reply text and URL selection from the candidate list only. Rule-based guards run first: legal/compliance questions are declined, and vague first-turn queries trigger a single clarifying question with an empty recommendation list. After the LLM responds, a validator drops any URL not in the catalog and caps the list at ten items. Targeted domain rules (leadership stack, Rust hiring, contact centre, graduate battery, etc.) improve Recall@10 on the public sample traces without hard-coding full conversation scripts.

5. Evaluation

I parsed gold URLs from the provided C1-C10 sample conversations and built a replay harness (`eval/replay.py`) that posts each user turn to `/chat` against a running service. On the deployed Render endpoint, mean Recall@10 across C1-C10 was 0.946. I also checked schema compliance, catalog-only URLs, vague-turn behaviour, and legal refusal on C7. Typical warm `/chat` latency was under 15 seconds, within the 30-second limit.

6. Deployment and trade-offs

The service runs on Render via Docker. CPU-only PyTorch and a pre-built FAISS cache were required to avoid out-of-memory failures on the free tier. I bundled `data/catalog.json` so cold start does not depend on the external catalog URL. Sonnet was chosen over smaller models because multi-turn clarification and refinement quality mattered more than marginal token savings.

7. Tools

Implementation: Python, FastAPI, FAISS, sentence-transformers, Anthropic API. Cursor was used for development assistance; sample conversations were used as behavioural reference, not as fixed scripts.